

Fundamentals of Fog and Edge Computing

BCSE313L

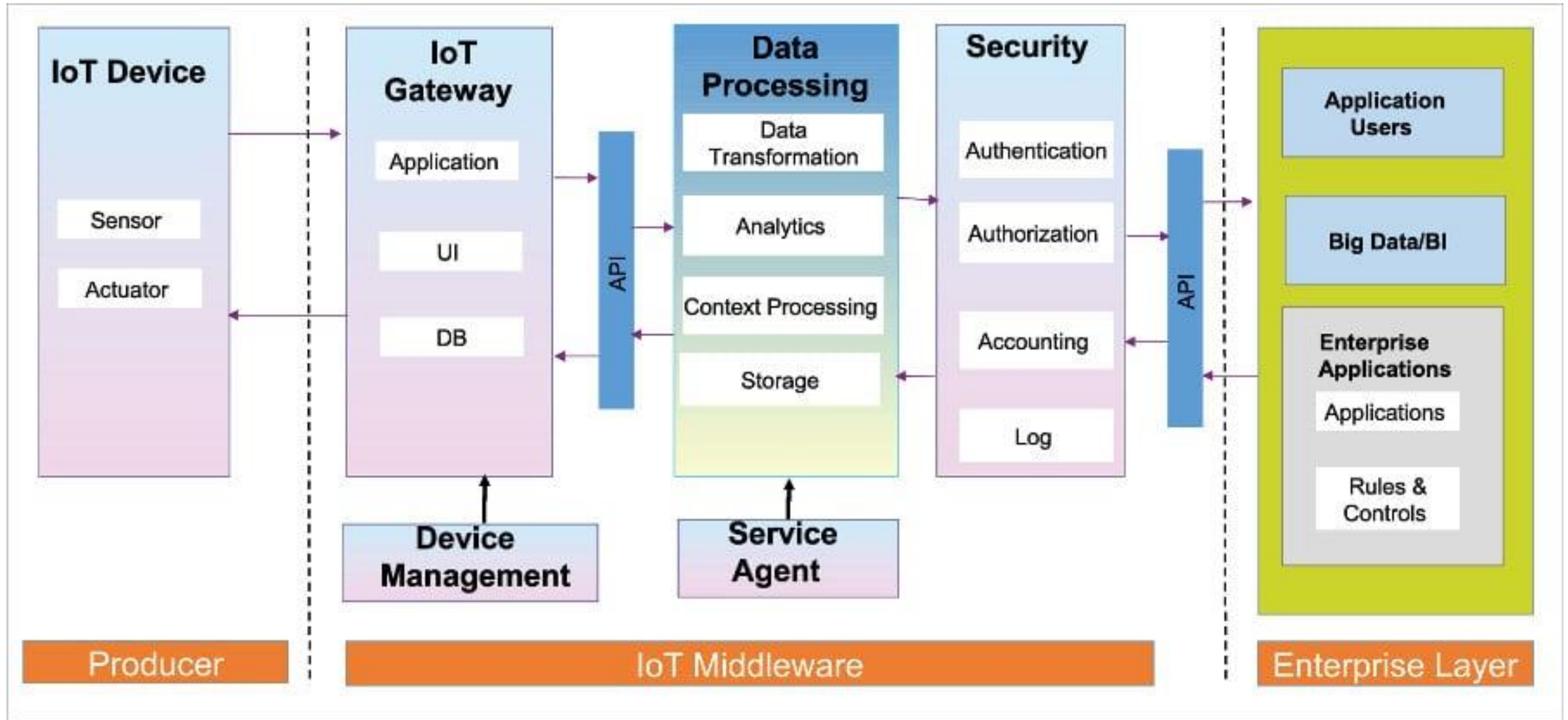
WINTER 2024 - 2025

Module 5 - Middleware for Fog and Edge Computing

Middleware – Definition & Purpose

- It serves as an intermediary layer that facilitates communication and data management between devices and applications.
- It is a software layer that connects different applications or services, enabling them to communicate and manage data effectively.
- Middleware simplifies the design of distributed mobile applications
- It reduces the complexity involved in developing applications that operate in dynamically changing environments

Middleware – Definition & Purpose



Middleware – Definition & Purpose

➤ The Things

- **IoT Device**: This is the foundational element, representing any physical device that can collect and transmit data. Examples include sensors, actuators, smart meters, wearable devices, etc.
- **Sensor**: Specifically, this indicates a component that detects and measures physical phenomena (e.g., temperature, pressure, light, motion) and converts it into a signal.
- **Actuator**: This component performs an action or controls a physical device based on received data or instructions (e.g., turning a motor on/off, adjusting a valve).
- **Producer**: The general term indicating the source of the data (the IoT device itself).

Middleware – Definition & Purpose

➤ The Network and Edge Processing

- **IoT Gateway:** This acts as an intermediary between the IoT devices and the cloud (or other central system). It often provides connectivity, local processing, data aggregation, and security.
- **Device Management:** This function is crucial for managing and controlling the IoT devices, including provisioning, configuration, updates, monitoring, and fault management.
- **Service Agent:** A software component running on the gateway or edge server that interacts with the devices and the platform. It could handle tasks like data translation, protocol conversion, and local data processing.
- **Application, UI, DB:** These components within the gateway suggest localized applications running on the edge, a user interface for local monitoring or control, and a local database for temporary data storage or caching.

Middleware – Definition & Purpose

➤ Data Processing and Security

➤ **Data Processing:** This block encompasses various data-related functions:

➤ **Data Transformation:** Converting data into a consistent and usable format.

➤ **Analytics:** Analyzing data to gain insights, identify patterns, and make predictions.

➤ **Context Processing:** Adding contextual information to the data to make it more meaningful.

➤ **Storage:** Persisting the data for later use and analysis.

Middleware – Definition & Purpose

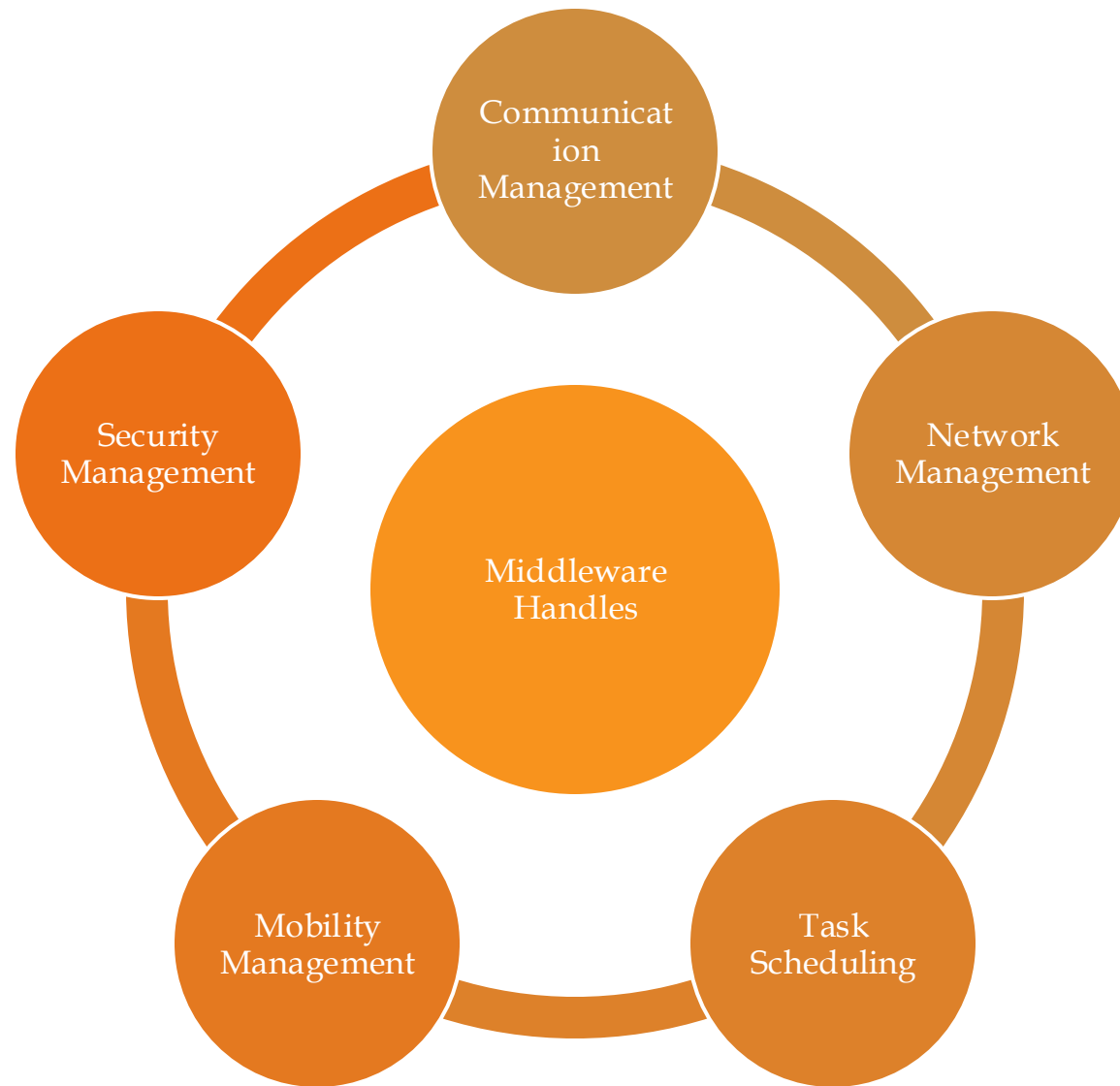
➤ Data Processing and Security

- **Security**: Security is paramount in IoT systems and includes:
 - **Authentication**: Verifying the identity of devices and users.
 - **Authorization**: Controlling access to resources and data based on roles and permissions.
 - **Accounting**: Tracking data access and usage for billing or auditing.
 - **Log**: Maintaining records of events and activities for security analysis and troubleshooting.

Middleware – Definition & Purpose

- **API (Application Programming Interface):**
- APIs are crucial for **enabling communication and data exchange between different components and systems**, such as between the gateway and the cloud platform, or between different microservices within the platform.

Middleware – Purpose



Middleware –Purpose/Need

Communication Management :

- Ensures seamless communication between different devices and layers within the fog and edge architecture.

Network Management :

- Manages network resources and optimizes data flow.

Task Scheduling :

- Allocates tasks efficiently across available resources.

Mobility Management :

- Handles the dynamic nature of mobile devices and ensures continuous service.

Security Management :

- Provides mechanisms for secure data transfer and access control.

Middleware – Communication Management

Device Discovery

Allowing new devices to join or leave the network dynamically (e.g., using protocols like MQTT for lightweight messaging).

Establishing Communication Channels

Setting up channels between requesting devices and ad-hoc discovered devices for data acquisition and processing.

Maintaining Communication

Ensuring continuous communication between devices, even in unreliable or low-bandwidth networks.

Message Exchange

Managing secure, scalable, and reliable message exchange between devices.

Middleware – Network Management

Multitier Network Distribution

Using software-defined networking (SDN) or virtual network topologies to distribute applications across fog, edge, and cloud layers

Connection Monitoring

Monitoring device connections and triggering procedures to resume lost connections.

Point-to- Point Networking

Managing connections between devices using technologies like TCP sockets, Wi-Fi, Wi-Fi Direct, or Bluetooth

Bandwidth Optimization

Minimizing bandwidth usage while ensuring efficient data transfer across the network.

Middleware – Task Scheduling & Management

Surrogate Selection

Choosing appropriate devices to execute tasks based on policies like **energy-aware selection, delay tolerance, or context-aware selection**

Resource Monitoring

Continuously monitoring the availability of resources (e.g., new user devices, VMs in fog or cloud layers)

Dynamic Task Assignment

Matching incoming tasks with suitable devices based on real-time requirements (e.g., GPS location, battery level, communication bandwidth).

Optimized Task Distribution

Distributing FEA tasks among multiple edge devices to minimize latency and resource usage.

Middleware – Mobility Management

Follow Me Cloud (FMC)

Ensuring middleware services and data follow mobile devices as they move (e.g., using Locator/ID separation protocols like LISP).

Service Continuity

Guaranteeing uninterrupted service in highly dynamic scenarios, such as vehicle-to-vehicle systems.

VM Migration

Managing the migration of virtual machines (VMs) across edge and fog nodes to accommodate device mobility.

Dynamic Device Tracking

Updating the system as devices change locations or contexts (e.g., using GPS or accelerometer data).

Middleware – Security Management

Authentication

Authenticating users and devices to prevent unauthorized access.

Using lightweight mutual authentication schemes for roaming fog nodes.

Privacy Protection

Ensuring data privacy for user devices through anonymization or encryption techniques.

Implementing policy-based access control mechanisms.

Encryption

Encrypting data during communication to ensure confidentiality.

Using crypto processors or lightweight encryption methods to reduce computation overhead.

Secure Offloading

Safely offloading application tasks to other edge devices for outsourcing.

Middleware – Design Challenges

Availability of Context on Sensing Devices

- Middleware must ensure that sensing devices have the necessary contextual information to perform tasks effectively.

Cost of Data Transfer and Processing

- The cost associated with transferring and processing data across different tiers of the Fog and Edge Architecture (FEA) needs to be minimized.

Limitations on Number of Edge Devices

- There are constraints on the number of edge devices available, along with dynamic changes in their context and mobility.

Strict Latency Constraints

- Many applications have strict real-time response requirements that cannot be met if processing is done far from the edge.

Middleware – Design Challenges

- **Challenge:** Ensuring that sensing devices have the necessary contextual information to perform tasks effectively.
- **Example:** Imagine a smart traffic light system in a city. The traffic lights need to adjust their signals based on real-time traffic conditions. For this, they rely on data from various sensors like cameras and vehicle detectors placed at intersections. If these sensors fail to provide accurate or timely context (e.g., the number of vehicles waiting at a red light), the traffic lights might not change signals optimally, leading to traffic congestion.

Middleware – Design Challenges

- **Challenge** Managing the **cost associated** with **transferring and processing data across different tiers** of the Fog and Edge Architecture (FEA).
- **Example :** Consider a large-scale agricultural setup where numerous **IoT sensors monitor soil moisture levels across a vast farm**. Sending all this raw data to a central cloud server for processing would require significant bandwidth and incur high costs. Instead, by **processing the data locally at edge devices and only sending summarized reports** (e.g., average moisture levels per zone) to the cloud, the **overall data transfer cost is minimized**.

Middleware – Design Challenges

- **Challenge :** Handling the **dynamic nature of edge devices**, including their **context and mobility**.
- **Example :** Think about a **fleet of delivery drones**. These **drones are mobile and constantly changing locations**. Middleware must dynamically adapt to these changes. For instance, if a **drone moves out of the range of one edge server**, it should **seamlessly connect to another nearby server without interrupting its mission**. This requires **continuous monitoring and adaptive management** of device contexts like location and network connectivity.

Middleware – Design Challenges

- **Challenge:** Meeting real-time response needs for latency-sensitive applications.
- **Example:** Consider an **autonomous vehicle system** that uses edge computing for real-time decision-making. The **system relies on data from multiple sensors** (cameras, LIDAR, etc.) to **detect obstacles and make driving decisions**. If the **processing of this sensor data is delayed** due to high latency, the **vehicle might not respond quickly enough to avoid a collision**. Processing the data at the edge, close to the source, ensures that decisions are made within the required time frame (e.g., milliseconds).

Middleware – Design Goals

Design Goals	Descriptions	Example
Ad-Hoc Device Discovery	Dynamically discover and establish communication with edge devices.	Smart traffic lights discovering nearby sensors for traffic monitoring.
Run-Time Execution Environment	Provide a platform for remote code execution on edge devices.	Offloading face detection tasks to a nearby fog node.
Minimal Task Disruption	Minimize interruptions during task execution due to mobility	network issues
Overhead of Operational Parameters	Minimize bandwidth and energy usage during middleware operations.	Transmitting only necessary sensor data to reduce bandwidth consumption.
Context-Aware Adaptive Design	Adapt to dynamic changes in device context and environment.	Adjusting app behaviour based on user activity (e.g.

Middleware – Design Goals

➤ Ad-Hoc Device Discovery

- Devices in fog/edge environments can range from IoT sensors, mobile devices, and fixed sensors to more powerful fog servers. **Middleware must facilitate the discovery of these devices dynamically.**

➤ Why It Matters :

- Applications often require **data acquisition and processing from multiple devices**. A **communication channel must be established between requesting devices and ad-hoc discovered devices**.
- Since **edge devices are dynamic** (they can join or leave the network frequently), **middleware must support a flexible communication layer** to enable seamless interaction.

➤ Example

- In a smart traffic system, a traffic light controller might need to discover nearby sensors or cameras to monitor traffic flow. Middleware ensures that these devices can communicate effectively.

Middleware – Design Goals

➤ Run-Time Execution Environment

➤ Middleware provides a **platform to execute application tasks remotely on edge devices**.

➤ **Code Download** : Middleware allows the application code to be downloaded to edge devices.

➤ **Remote Execution** : The downloaded code is executed locally on the edge device.

➤ **Result Delivery** : Results are sent back to the requesting device or centralized server.

➤ Why It Matters :

➤ Many applications require real-time processing at the edge to reduce latency. For example, a video surveillance system may offload face detection tasks to nearby edge devices instead of sending raw video streams to the cloud.

➤ Example:

➤ A drone monitoring system could offload image processing tasks to a nearby fog node for faster analysis.

Middleware – Design Goals

➤ Minimal Task Disruption

- Ensure that tasks **running on edge devices are not interrupted** unnecessarily.

➤ Challenges Addressed

- **Device Mobility** : Devices may move out of range or lose connectivity.
- **Resource Constraints** : Devices may run out of battery or processing power.
- **Network Disconnections** : Unreliable networks can disrupt task execution.

- **Solution** : Middleware uses **anticipatory techniques to predict changes in device context** (e.g., battery level, location) and **intelligently reschedule tasks to minimize disruption**.

➤ Why It Matters :

- Disruptions can lead to **reinitialization of tasks or incomplete results**, which affects application reliability.

➤ Example

- In a connected vehicle system, if a car moves out of the range of one fog node, middleware ensures that the task is handed over to another nearby node without interruption.

Middleware – Design Goals

➤ Overhead of Operational Parameters

- **Minimize the additional bandwidth and energy consumption** incurred during middleware operations.

➤ Challenges Addressed

- **Bandwidth Usage** : Establishing communication channels, selecting candidate devices, and distributing tasks across multiple devices consume bandwidth.
- **Energy Consumption** : Edge devices often have limited battery life, so excessive computation or communication can drain resources quickly.

➤ **Solution** Middleware optimizes resource usage by:

- Selecting devices with sufficient resources.
- Distributing tasks efficiently to avoid overloading any single device.

➤ **Why It Matters** :

- Edge devices are resource-constrained, so minimizing operational overhead is critical for efficient task execution.

➤ **Example**

- In a smart agriculture system, middleware ensures that only necessary sensor data is transmitted to the fog server, reducing bandwidth usage.

Middleware – Design Goals

➤ Context-Aware Adaptive Design

- Middleware must **adapt to dynamic changes** in device context and the environment.
- **Context Monitoring** : Continuously monitors **contextual information** such as device location, battery level, and user activity.
- **Self-Adaptive Services** : Middleware **adjusts its behavior based on changes** in context to improve performance and quality of service (QoS).
- **Why It Matters** :
 - Applications often rely on real-time contextual data (e.g., user location, environmental conditions) to function effectively.
- **Example**
 - A fitness app might use context-aware middleware to adjust its recommendations based on the user's current activity (e.g., walking, running).

Middleware – Design Goals

➤ Quality of Service (QoS)

- Ensure that **middleware meets the specific QoS requirements** of applications.
- **Real-Time Response** : Applications like autonomous vehicles or emergency response systems require ultra-low latency.
- **Data Relevance** : Acquired data must be relevant and accurate for the application.
- **Uninterrupted Data Acquisition** : Middleware must ensure continuous data collection even in dynamic environments.
- **Why It Matters** :
 - Different applications have different QoS needs. For example, a healthcare monitoring system may prioritize accuracy and reliability, while a video streaming app may prioritize low latency.
- **Example**
 - In a disaster response system, middleware ensures that critical data (e.g., location of survivors) is collected and processed in real time.

State-of-the-Art Middleware Infrastructures

Middleware supports real-time data streaming applications such as:

- Traffic monitoring (e.g., Waze, smart traffic light systems)
- Real-time replay in stadiums
- Video analytics
- Emergency rescue and missing persons search
- Smart vehicle-to-vehicle systems

State-of-the-Art Middleware Infrastructures



State-of-the-Art Middleware Infrastructures

➤ Recent Middleware Solutions

- FemtoCloud (Supports data analytics and optimized device selection)
- Nakamura et al. (Proposes in-device processing for IoT)
- CloudAware (Adapts dynamically to changing network conditions)
- MEC-based and Fog-based middleware (Such as Carrega et al., Grewe et al.)

State-of-the-Art Middleware Infrastructures

Middleware Approaches –
Some recent works
propose:

- **Microservices**-based middleware for IoT (e.g., Carrega et al.)
- **Service-oriented middleware** for cloud and fog processing
- **Context-aware middleware** (MobiPADs, MobiCon) that adapts to mobile changes
- **Cloudlet-based solutions** for real-time response and offloading

State-of-the-Art Middleware Infrastructures

➤ **Microservice Approach**

➤ Middleware is designed as a collection of independent, lightweight microservices that interact via APIs.

➤ **Example:** Carrega et al. propose a microservices-based middleware where different services (e.g., authentication, scheduling, data analytics) can be dynamically scaled and deployed in fog nodes.

➤ **Advantages:**

➤ **Scalability:** Components can be deployed separately based on demand.

➤ **Modularity:** Easy to update or replace individual services.

➤ **Flexibility:** Supports heterogeneous edge environments.

State-of-the-Art Middleware Infrastructures

➤ **Service-Oriented Middleware (SOM)**

- Provides a **service-based framework** where applications interact with middleware through well-defined **service interfaces**.

➤ **Example: Global Sensor Networks (GSN)** is a widely used middleware that processes data from distributed sensors using a service-oriented approach.

➤ **Key Features:**

- **Interoperability:** Applications can use standard service calls (SOAP, REST).
- **Dynamic Composition:** Services can be composed dynamically based on application needs.
- **Scalability:** Can be deployed across multiple edge/fog nodes.

State-of-the-Art Middleware Infrastructures

➤ Context-Aware Middleware

- Middleware adapts dynamically based on real-time contextual changes (e.g., user location, device movement, network conditions).

➤ Example:

- MobiPADs – A context-aware middleware that adjusts mobile services based on sensor inputs.
- MobiCon – Middleware that dynamically reconfigures network connections based on mobility patterns.

➤ Advantages:

- Personalized User Experience: Adjusts behavior based on device/user conditions.
- Efficiency: Reduces unnecessary resource consumption by predicting user needs.
- Resilience: Handles dynamic changes in the edge environment.

State-of-the-Art Middleware Infrastructures

➤ **Cloudlet-Based Middleware**

- Uses cloudlets (small-scale, low-latency cloud nodes close to edge devices) to handle computing and storage.

➤ **Example:**

- **CloudAware** – Middleware designed for adaptive cloudlet computing, allowing edge devices to seamlessly offload tasks.
- Satyanarayan et al. proposed a "data center in a box" model using cloudlets for real-time processing.

➤ **Advantages:**

- **Low Latency:** Reduces dependency on remote cloud servers.
- **Energy Efficiency:** Minimizes energy consumption on edge devices.
- **Support for Mobile Users:** Migrates tasks dynamically across cloudlets.

Middleware – Proposed Architecture

Middleware Services in Fog Edge Computing

API Code

Security

Device
Discovery

Authentication

Privacy

Encryption

Middleware – Proposed Architecture

Middleware Components in Fog Edge Computing

Context
Monitoring
and
Prediction

Selection of Participating Devices

Data
Analytics

Scheduling
and Resource
Management

Network
Management

Execution
Management

Mobility
Management

Energy-
Aware
Selection

Delay
Tolerance-
Based
Selection

Context-
Aware
Selection

Middleware – Case Study 1

- **Scenario:** A smart city has traffic cameras, sensors, and fog servers to monitor congestion.
- **Problem:** If all cameras send raw data to a central cloud, there will be delays, network congestion, and high costs.

Middleware – Case Study 1

- Solution is to use Fog Computing (With Middleware)

Device
Discovery

Task
Scheduling

Security

Data Filtering

Mobility
Support

Middleware – Case Study 1

Device Discovery

- Middleware finds nearby cameras and traffic sensors.

Data Filtering

- Instead of sending all video footage to the cloud, the middleware processes images locally and only sends critical alerts (e.g., accident detection) to authorities.

Task Scheduling

- If one fog server is too busy, middleware assigns the task to another less-loaded server.

Mobility Support

- If a vehicle is moving between different zones, the middleware ensures continuous data tracking across edge devices.

Security

- Middleware ensures only authorized devices can access sensitive traffic data.

Middleware – Case Study 2

- Imagine you have a smart thermostat in your home that automatically adjusts the room temperature based on your preferences, weather conditions, and room occupancy.
- If the thermostat relied solely on cloud computing, it would have to send temperature data to the cloud every second, wait for the cloud to process the data, and then receive instructions on whether to turn the heater or AC on/off. This would cause:
 - Delays in response time.
 - Increased internet usage, leading to higher bandwidth costs.
 - Dependence on cloud connectivity—if your internet goes down, the thermostat might stop working efficiently.
- Which types of Middleware Enhances Smart Thermostat Functionality with Fog and Edge Computing

Middleware – Case Study 2

Context-Aware Middleware

- Detects the current room conditions and user behavior to optimize thermostat operation
- Detects room occupancy (using motion sensors) and adjusts temperature settings accordingly.
- Uses weather forecast data to pre-adjust heating/cooling before drastic temperature changes.

Data Filtering Middleware

- Filters unnecessary data to reduce cloud communication and bandwidth usage.
- Instead of sending constant temperature readings to the cloud, the middleware only sends data when temperature changes significantly.
- Aggregates temperature data from multiple rooms and sends only relevant information.

Middleware – Case Study 2

Device Discovery Middleware

- Detects and connects the thermostat with other smart home devices (e.g., motion sensors, smart blinds, and weather sensors).
- Identifies nearby devices (e.g., motion sensors) to determine if someone is in the room.
- Connects with smart blinds to check if sunlight is heating the room.

Task Scheduling Middleware

- Assigns tasks to the right processing unit (thermostat, fog server, or cloud).
- If the thermostat's processor is busy, middleware offloads computations to a local fog node instead of the cloud.
- Balances processing between multiple smart home devices to avoid delays.

Middleware – Case Study 2

Security & Privacy Middleware

- Ensures secure data transmission and protects thermostat data from cyber threats.
- Encrypts temperature and user data before sending it to the cloud.
- Prevents unauthorized access to thermostat controls.

Automation Middleware

- Enables seamless automation by connecting multiple smart home devices.
- Triggers temperature adjustments based on input from other smart home devices.
- If the smart lock detects that no one is home, middleware switches the thermostat to eco mode.